

Защищено:
Большаков С.А.

Демонстрация ЛР:
Большаков С.А.

"_ " _____ 2025 г.

"_ " _____ 2025 г.

**Отчет по лабораторной работе № 4 по курсу
Системное программирование**

" Циклы и перевод символов "

(есть ли дополнительные требования - ДА)

(Обеспечен контроль ввода значения символа)

15
(количество листов)
Вариант № <10>

ИСПОЛНИТЕЛЬ:

студент группы ИУ5-42Б

Кудабаева Э.

(подпись)

"23" марта 2025 г.

СОДЕРЖАНИЕ

1. Цель выполнения лабораторной работы № 4.....	3
2. Порядок и условия проведения работы № 4	3
3. Описание ошибок, возникших при отладке № 4	3
4. Блок-схема программы.....	3
5. Скриншот программы КН № 4 в TD.exe	4
6. Текст программы на языке Ассемблера	5
7. Результаты работы программы.....	9
8. Выводы по ЛР № 4.....	15

1. Цель выполнения лабораторной работы № 4

Лабораторная работа №4 выполняется для получения навыков разработки и отладки программ на ЯП, получения базовых знаний об использовании циклов и перевода символов из одной системы в другую на Ассемблере, изучения и использования компонентов системы программирования Ассемблер (компилятора, редактора связей, отладчика) и получения навыков оформления документации по программным разработкам, реализуемым на языке.

2. Порядок и условия проведения работы № 4

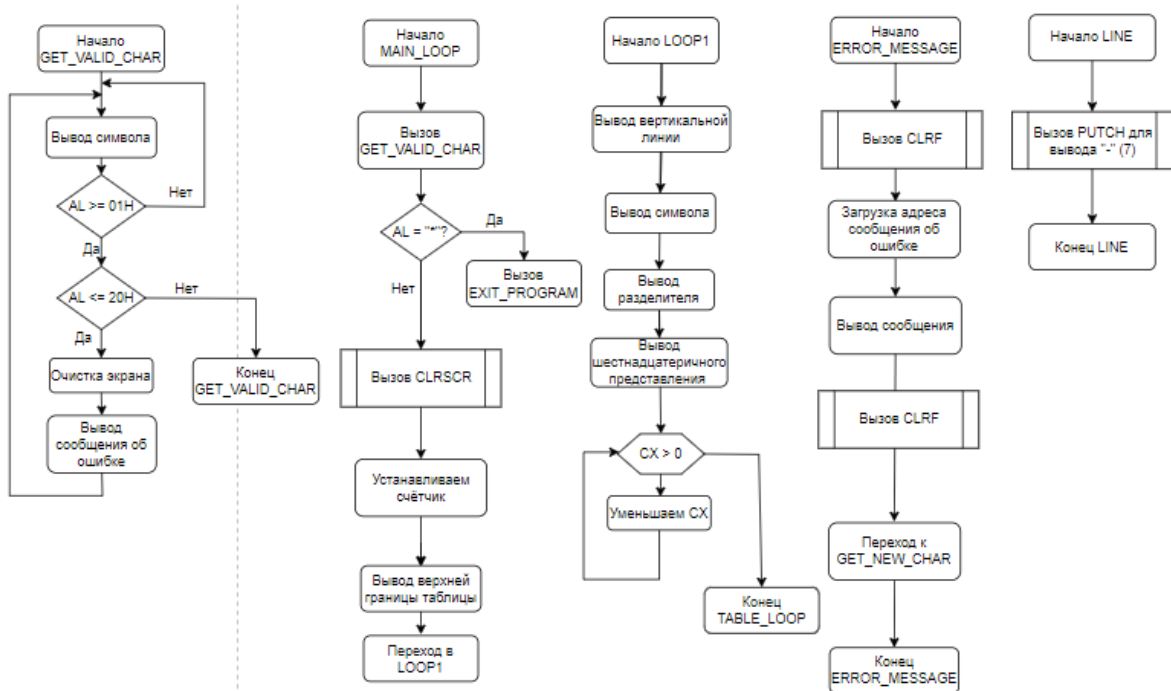
Разработать и отладить циклическую программу на языке Ассемблер для вывода на экран 20 последовательных прописных букв русского алфавита (начиная с символа “А” или другого символа, введенного с клавиатуры).

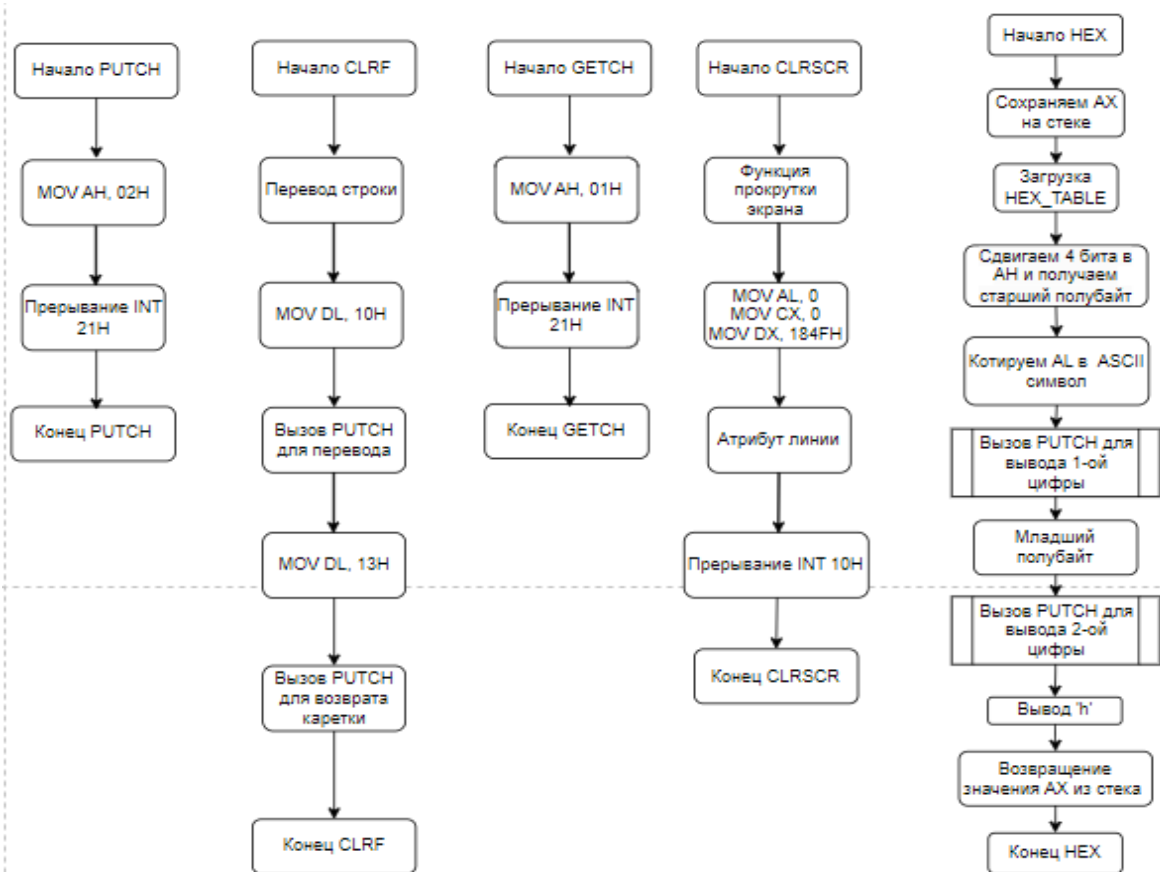
После завершения вывода букв и их кодов на экран организовать ожидание ввода любого символа с клавиатуры (нажатие клавиши).

3. Описание ошибок, возникших при отладке № 4

Ошибки отсутствуют.

4. Блок-схема программы





5. Скриншот программы КН № 4 в TD.exe

The screenshot shows the TD debugger interface. The main window displays assembly code for a program named 'lab4_dop'. The code includes comments in Russian and assembly instructions. The registers window on the right shows the current state of the CPU registers (80486). The status bar at the bottom indicates the current instruction pointer (IP) and other debugging information.

Module: lab4_dop	File: =1[+][+]	CPU 80486	3
#lab4_dop#start: PUS	ax 0000	c=0	
cs:003B push cs	bx 0000	z=0	
#lab4_dop#16: POP DS	cx 0000	s=0	
cs:003C pop ds	dx 0000	o=0	
#lab4_dop#17: CALL C	si 0000	p=0	
cs:003D call #lab	di 0000	a=0	
#lab4_dop#main_loop:	bp 0000	i=1	
cs:0040 call #lab	sp 0000	d=0	
#lab4_dop#21: CMP AL	ds 5569		
cs:0043 cmp al,2	es 5569		
#lab4_dop#22: JE EXI	ss 5578		
cs:0045 je #lab	cs 5579		
#lab4_dop#23: MOV IN	ip 003B		
ds:0000 CD 20 FF 9F			
ds:0008 AD DE E0 01			
ds:0010 11 1C 89 02	ss:0002 16D8		
ds:0018 01 01 01 00	ss:0000 001A		

Watches: 2

F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu

(Примечание: к демонстрации ЛР 4 – исправить отладку – выполнено, сообщение см. в результатах вывода программы)

6. Текст программы на языке Ассемблера

; Лабораторная работа 4
; Кудабаява Э., ИУ5-42Б

```
MYCODE SEGMENT 'CODE'
    ASSUME CS:MYCODE, DS:MYCODE
```

; Данные

```
INPUT_CH DB ? ; Переменная для хранения введенного символа
ERROR_MSG DB 'НЕДОПУСТИМЫЙ СИМВОЛ!$' ; Сообщение о некорректном символе
EXIT_MSG DB 'Завершение программы$' ; Сообщение при завершении программы
HEX_TABLE DB '0123456789ABCDEF' ; Таблица для перевода в 16-ный формат
```

; Начало программы

START:

```
    PUSH CS
    POP DS ; Инициализация DS
    CALL CLRSCR ; Очистка экрана
```

MAIN_LOOP:

```
    CALL GET_VALID_CHAR ; Ввод символа
    CMP AL, '*' ; Проверка на символ '*'
    JE EXIT_PROGRAM ; Если введен '*', завершаем программу
    MOV INPUT_CH, AL ; Сохраняем введенный символ
```

; Очистка экрана и начало вывода таблицы

```
    CALL CLRSCR
    MOV CX, 20 ; Количество символов для вывода (20)
```

; Верхняя граница таблицы

```
    MOV DL, 218 ; Левый верхний угол
    CALL PUTCH
    CALL LINE ; Горизонтальная линия
    MOV DL, 191 ; Правый верхний угол
    CALL PUTCH
    CALL CLRF ; Переход на новую строку
```

; Основной цикл вывода символов

LOOP1:

```
    MOV DL, 179 ; Вертикальная линия
    CALL PUTCH
```

; Вывод символа

```
    MOV DL, INPUT_CH
    CALL PUTCH
```

; Вывод разделителя

```
    MOV DL, ' '
    CALL PUTCH
    MOV DL, '-'
    CALL PUTCH
```

```
MOV DL, ''
CALL PUTCH
```

```
; Вывод шестнадцатеричного представления
MOV AL, INPUT_CH
CALL HEX
```

```
MOV DL, 179      ; Вертикальная линия
CALL PUTCH
CALL CLRF        ; Переход на новую строку
```

```
; Разделительная линия
MOV DL, 195      ; Левый разделитель
CALL PUTCH
CALL LINE        ; Горизонтальная линия
MOV DL, 180      ; Правый разделитель
CALL PUTCH
CALL CLRF        ; Переход на новую строку
```

```
; Переход к следующему символу
INC INPUT_CH
LOOP LOOP1       ; Повтор цикла
```

```
; Нижняя граница таблицы
```

```
LAST_LET:
MOV DL, 192      ; Левый нижний угол
CALL PUTCH
CALL LINE        ; Горизонтальная линия
MOV DL, 217      ; Правый нижний угол
CALL PUTCH
CALL CLRF        ; Переход на новую строку
```

```
; Ожидание ввода для продолжения
CALL GETCH
JMP MAIN_LOOP   ; Возврат к началу программы
```

```
; Завершение программы
```

```
EXIT_PROGRAM:
LEA DX, EXIT_MSG ; Сообщение о завершении
MOV AH, 09H
INT 21H

MOV AH, 4CH      ; Завершение программы
MOV AL, 5
INT 21H
```

```
; Процедура ввода допустимого символа с проверкой диапазона
```

```
GET_VALID_CHAR PROC NEAR
```

```
GET_NEW_CHAR:
CALL GETCH       ; Ввод символа
CMP AL, 01H
JGE CHECK_UPPER
```

```
;CMP AL, 20H      ; Проверка, >= 20H
;JGE VALID_CHAR    ; Если да, символ допустим
;CMP AL, 80H      ; Проверка, >= 80H
;JGE VALID_CHAR    ; Если да, символ допустим
```

```
VALID_CHAR:
    RET
GET_VALID_CHAR ENDP
```

```
; Процедура ввода символа
GETCH PROC NEAR
    MOV AH, 00H    ; Ожидание ввода символа с клавиатуры
    INT 16H        ; BIOS-функция
    RET
GETCH ENDP
```

```
CHECK_UPPER PROC NEAR
    CMP AL, 20H
    JLE ERROR_SIGN
    JMP VALID_CHAR
```

```
CHECK_UPPER ENDP
```

```
ERROR_SIGN PROC NEAR
; Вывод сообщения об ошибке
    CALL CLRF
    LEA DX, ERROR_MSG
    MOV AH, 09H
    INT 21H

    CALL CLRF      ; Перевод строки
    JMP GET_NEW_CHAR ; Повторный ввод
ERROR_SIGN ENDP
```

```
; Процедура вывода символа
PUTCH PROC NEAR
    MOV AH, 02H
    INT 21H
    RET
PUTCH ENDP
```

```
; Процедура перевода строки
CLRF PROC NEAR
    MOV DL, 10      ; Перевод строки
    CALL PUTCH
    MOV DL, 13      ; Возврат каретки
    CALL PUTCH
    RET
CLRF ENDP
```

```
; Процедура очистки экрана
```

CLRSCR PROC NEAR

MOV AH, 06H ; Функция прокрутки экрана

MOV AL, 0 ; Очистка всего экрана

MOV CX, 0 ; Верхний левый

MOV DX, 184FH ; Нижний правый

MOV BH, 07H ; Атрибут

INT 10H

RET

CLRSCR ENDP

; Процедура рисования горизонтальной линии

LINE PROC NEAR

MOV DL, 196 ; Символ горизонтальной линии

CALL PUTCH

CALL PUTCH

CALL PUTCH

CALL PUTCH

CALL PUTCH

CALL PUTCH

CALL PUTCH

RET

LINE ENDP

; Процедура перевода символа в шестнадцатеричный формат

HEX PROC NEAR

PUSH AX

LEA BX, HEX_TABLE ; Загрузка таблицы перекодировки символов

MOV AH, AL

SHR AH, 4

MOV AL, AH

XLAT ; Преобразование с помощью XLAT

MOV DL, AL

CALL PUTCH

POP AX

PUSH AX

MOV AH, AL

SHL AH, 4

SHR AH, 4

MOV AL, AH

XLAT ; Преобразование с помощью XLAT

MOV DL, AL

CALL PUTCH

; Вывод 'h' для 16-ного формата

MOV DL, 'h'

CALL PUTCH

POP AX

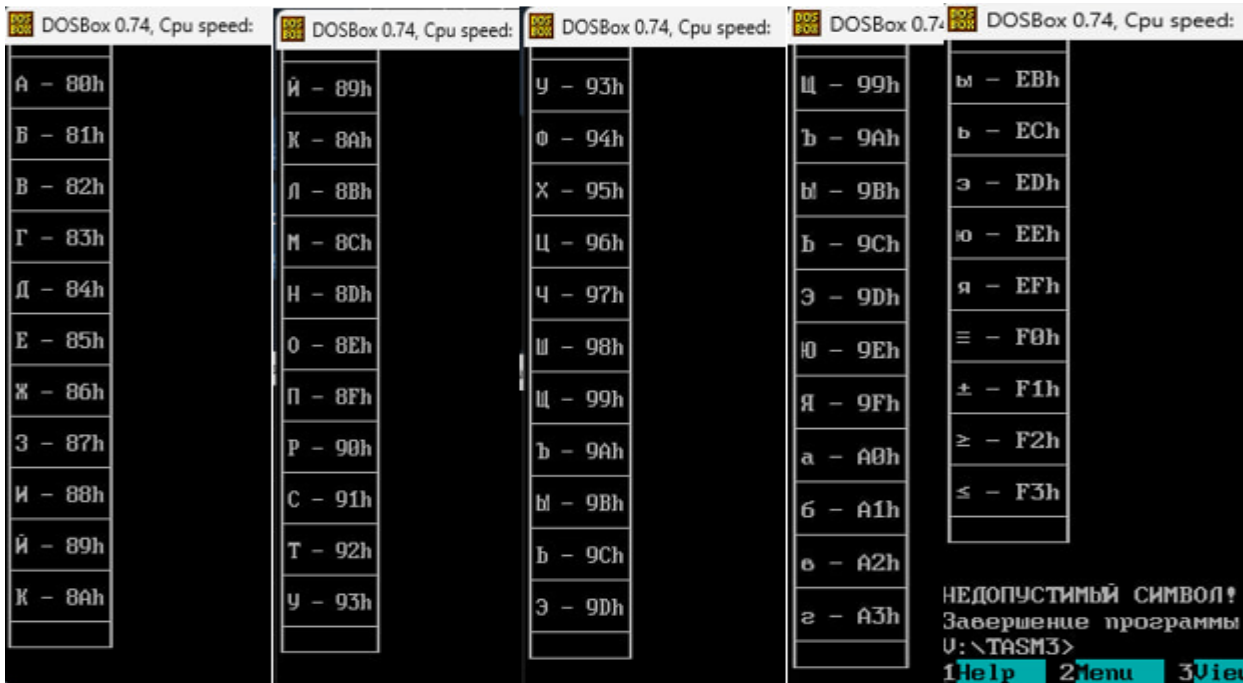
RET

HEX ENDP

MYCODE ENDS

END START

7. Результаты работы программы



Turbo Assembler Version 3.1 03/23/25 21:18:48 Page 1
lab4_dop.asm

```

1          ; Лабораторная работа 4
2          ; Кудабаева Э., ИУ5-42Б
3
4 0000      MYCODE SEGMENT 'CODE'
5           ASSUME CS:MYCODE, DS:MYCODE
6
7          ; Данные
8 0000 ??   INPUT_CH DB ?          ; Переменная для хранения введенного
символа
9 0001 8D 85 84 8E 8F 93 91+ ERROR_MSG DB 'НЕДОПУСТИМЫЙ СИМВОЛ!$' ;
Сообщение о некорректном символе
10 92 88 8C 9B 89 20 91+
11 88 8C 82 8E 8B 21 24
12 0016 87 A0 A2 A5 E0 E8 A5+ EXIT_MSG DB 'Завершение программы$' ; Сообщение
при завершении программы
13 AD A8 A5 20 AF E0 AE+
14 A3 E0 A0 AC AC EB 24
15 002B 30 31 32 33 34 35 36+ HEX_TABLE DB '0123456789ABCDEF' ; Таблица для
перевода в 16-ный формат
16 37 38 39 41 42 43 44+
17 45 46
18

```

```

19                                     ; Начало программы
20 003B START:
21 003B 0E PUSH CS
22 003C 1F POP DS ; Инициализация DS
23 003D E8 00B3 CALL CLRSCR ; Очистка экрана
24
25 0040 MAIN_LOOP:
26 0040 E8 007E CALL GET_VALID_CHAR ; Ввод символа
27 0043 3C 2A CMP AL, '*' ; Проверка на символ '*'
28 0045 74 6D JE EXIT_PROGRAM ; Если введен '*', завершаем
программу
29 0047 A2 0000r MOV INPUT_CH, AL ; Сохраняем введенный
символ
30
31 ; Очистка экрана и начало вывода таблицы
32 004A E8 00A6 CALL CLRSCR
33 004D B9 0014 MOV CX, 20 ; Количество символов
для вывода (20)
34
35 ; Верхняя граница таблицы
36 0050 B2 DA MOV DL, 218 ; Левый верхний угол
37 0052 E8 008E CALL PUTCN
38 0055 E8 00AA CALL LINE ; Горизонтальная линия
39 0058 B2 BF MOV DL, 191 ; Правый верхний угол
40 005A E8 0086 CALL PUTCN
41 005D E8 0088 CALL CLRF ; Переход на новую строку
42
43 ; Основной цикл вывода символов
44 0060 LOOP1:
45 0060 B2 B3 MOV DL, 179 ; Вертикальная линия
46 0062 E8 007E CALL PUTCN
47
48 ; Вывод символа
49 0065 8A 16 0000r MOV DL, INPUT_CH
50 0069 E8 0077 CALL PUTCN
51
52 ; Вывод разделителя
53 006C B2 20 MOV DL, ''
54 006E E8 0072 CALL PUTCN
55 0071 B2 2D MOV DL, '-'
56 0073 E8 006D CALL PUTCN
57 0076 B2 20 MOV DL, ''
58 0078 E8 0068 CALL PUTCN
59
60 ; Вывод шестнадцатеричного представления
61 007B A0 0000r MOV AL, INPUT_CH
62 007E E8 0099 CALL HEX
63

```

```

64 0081 B2 B3          MOV DL, 179      ; Вертикальная линия
65 0083 E8 005D        CALL PUTCH
66 0086 E8 005F        CALL CLRF          ; Переход на новую строку
67
68                    ; Разделительная линия
69 0089 B2 C3          MOV DL, 195      ; Левый разделитель
70 008B E8 0055        CALL PUTCH
71 008E E8 0071        CALL LINE          ; Горизонтальная линия
72 0091 B2 B4          MOV DL, 180      ; Правый разделитель
73 0093 E8 004D        CALL PUTCH
74 0096 E8 004F        CALL CLRF          ; Переход на новую строку
75
76                    ; Переход к следующему символу
77 0099 FE 06 0000r    INC INPUT_CH
78 009D E2 C1          LOOP LOOP1        ; Повтор цикла
79
80                    ; Нижняя граница таблицы
81 009F                LAST_LET:
82 009F B2 C0          MOV DL, 192      ; Левый нижний угол
83 00A1 E8 003F        CALL PUTCH
84 00A4 E8 005B        CALL LINE          ; Горизонтальная линия
85 00A7 B2 D9          MOV DL, 217      ; Правый нижний угол
86 00A9 E8 0037        CALL PUTCH
87 00AC E8 0039        CALL CLRF          ; Переход на новую строку
88
89                    ; Ожидание ввода для продолжения
90 00AF E8 0017        CALL GETCH
91 00B2 EB 8C          JMP MAIN_LOOP      ; Возврат к началу программы
92
93                    ; Завершение программы
94 00B4                EXIT_PROGRAM:
95 00B4 BA 0016r        LEA DX, EXIT_MSG  ; Сообщение о завершении
96 00B7 B4 09          MOV AH, 09H
97 00B9 CD 21          INT 21H
98
99 00BB B4 4C          MOV AH, 4CH      ; Завершение программы
100 00BD B0 05          MOV AL, 5
101 00BF CD 21          INT 21H
102
103                    ; Процедура ввода допустимого символа с проверкой диапазона
104 00C1                GET_VALID_CHAR PROC NEAR
105 00C1                GET_NEW_CHAR:
106 00C1 E8 0005        CALL GETCH          ; Ввод символа
107 00C4 3C 01          CMP AL, 01H
108 00C6 7D 06          JGE CHECK_UPPER
109                    ; CMP AL, 20H      ; Проверка, >= 20H
110                    ; JGE VALID_CHAR    ; Если да, символ допустим
111                    ; CMP AL, 80H      ; Проверка, >= 80H
112                    ; JGE VALID_CHAR    ; Если да, символ допустим
113
114

```

Turbo Assembler Version 3.1 03/23/25 21:18:48 Page 3
lab4_dop.asm

```

115 00C8                VALID_CHAR:
116 00C8 C3            RET
117 00C9                GET_VALID_CHAR ENDP
118
119                    ; Процедура ввода символа
120 00C9                GETCH PROC NEAR
121 00C9 B4 00            MOV AH, 00H ; Ожидание ввода символа с клавиатуры
122 00CB CD 16            INT 16H ; BIOS-функция
123 00CD C3            RET
124 00CE                GETCH ENDP
125
126 00CE                CHECK_UPPER PROC NEAR
127 00CE 3C 20            CMP AL, 20H
128 00D0 7E 02            JLE ERROR_SIGN
129 00D2 EB F4            JMP VALID_CHAR
130
131 00D4                CHECK_UPPER ENDP
132
133 00D4                ERROR_SIGN PROC NEAR
134                    ; Вывод сообщения об ошибке
135 00D4 E8 0011          CALL CLRF
136 00D7 BA 0001r        LEA DX, ERROR_MSG
137 00DA B4 09            MOV AH, 09H
138 00DC CD 21            INT 21H
139
140 00DE E8 0007          CALL CLRF ; Перевод строки
141 00E1 EB DE            JMP GET_NEW_CHAR ; Повторный ввод
142 00E3                ERROR_SIGN ENDP
143
144                    ; Процедура вывода символа
145 00E3                PUTCH PROC NEAR
146 00E3 B4 02            MOV AH, 02H
147 00E5 CD 21            INT 21H
148 00E7 C3            RET
149 00E8                PUTCH ENDP
150
151                    ; Процедура перевода строки
152 00E8                CLRFL PROC NEAR
153 00E8 B2 0A            MOV DL, 10 ; Перевод строки
154 00EA E8 FFF6          CALL PUTCH
155 00ED B2 0D            MOV DL, 13 ; Возврат каретки
156 00EF E8 FFF1          CALL PUTCH
157 00F2 C3            RET
158 00F3                CLRFL ENDP
159
160                    ; Процедура очистки экрана
161 00F3                CLRSCR PROC NEAR
162 00F3 B4 06            MOV AH, 06H ; Функция прокрутки экрана
163 00F5 B0 00            MOV AL, 0 ; Очистка всего экрана

```

```

164 00F7 B9 0000          MOV CX, 0          ; Верхний левый
165 00FA BA 184F          MOV DX, 184FH ; Нижний правый
166 00FD B7 07            MOV BH, 07H    ; Атрибут
167 00FF CD 10            INT 10H
168 0101 C3              RET
169 0102                CLRSCR ENDP
170
171                    ; Процедура рисования горизонтальной линии

```

Turbo Assembler Version 3.1 03/23/25 21:18:48 Page 4
lab4_dop.asm

```

172 0102                LINE PROC NEAR
173 0102 B2 C4            MOV DL, 196    ; Символ горизонтальной линии
174 0104 E8 FFDC          CALL PUTCH
175 0107 E8 FFD9          CALL PUTCH
176 010A E8 FFD6          CALL PUTCH
177 010D E8 FFD3          CALL PUTCH
178 0110 E8 FFD0          CALL PUTCH
179 0113 E8 FFCD          CALL PUTCH
180 0116 E8 FFCA          CALL PUTCH
181 0119 C3              RET
182 011A                LINE ENDP
183
184                    ; Процедура перевода символа в шестнадцатеричный формат
185 011A                HEX PROC NEAR
186 011A 50              PUSH AX
187 011B BB 002Br          LEA BX, HEX_TABLE ; Загрузка таблицы
перекодировки символов
188
189 011E 8A E0            MOV AH, AL
190 0120 D0 EC D0 EC D0 EC D0+ SHR AH, 4
191    EC
192 0128 8A C4            MOV AL, AH
193 012A D7              XLAT          ; Преобразование с помощью XLAT
194 012B 8A D0            MOV DL, AL
195
196 012D E8 FFB3          CALL PUTCH
197
198 0130 58              POP AX
199 0131 50              PUSH AX
200 0132 8A E0            MOV AH, AL
201 0134 D0 E4 D0 E4 D0 E4 D0+ SHL AH, 4
202    E4
203 013C D0 EC D0 EC D0 EC D0+ SHR AH, 4
204    EC
205 0144 8A C4            MOV AL, AH
206 0146 D7              XLAT          ; Преобразование с помощью XLAT
207 0147 8A D0            MOV DL, AL
208 0149 E8 FF97          CALL PUTCH
209
210                    ; Вывод 'h' для 16-ного формата

```

```

211 014C B2 68          MOV DL, 'h'
212 014E E8 FF92        CALL PUTCH
213
214 0151 58            POP AX
215 0152 C3            RET
216 0153              HEX ENDP
217
218 0153              MYCODE ENDS
219                  END START

```

Turbo Assembler Version 3.1 03/23/25 21:18:48 Page 5
Symbol Table

Symbol Name	Type	Value
??DATE	Text	"03/23/25"
??FILENAME	Text	"lab4_dop"
??TIME	Text	"21:18:48"
??VERSION	Number	030A
@CPU	Text	0101H
@CURSEG	Text	MYCODE
@FILENAME	Text	LAB4_DOP
@WORDSIZE	Text	2
CHECK_UPPER	Near	MYCODE:00CE
CLRF	Near	MYCODE:00E8
CLRSCR	Near	MYCODE:00F3
ERROR_MSG	Byte	MYCODE:0001
ERROR_SIGN	Near	MYCODE:00D4
EXIT_MSG	Byte	MYCODE:0016
EXIT_PROGRAM	Near	MYCODE:00B4
GETCH	Near	MYCODE:00C9
GET_NEW_CHAR	Near	MYCODE:00C1
GET_VALID_CHAR	Near	MYCODE:00C1
HEX	Near	MYCODE:011A
HEX_TABLE	Byte	MYCODE:002B
INPUT_CH	Byte	MYCODE:0000
LAST_LET	Near	MYCODE:009F
LINE	Near	MYCODE:0102
LOOP1	Near	MYCODE:0060
MAIN_LOOP	Near	MYCODE:0040
PUTCH	Near	MYCODE:00E3
START	Near	MYCODE:003B
VALID_CHAR	Near	MYCODE:00C8

Groups & Segments	Bit	Size	Align	Combine	Class
MYCODE	16	0153	Para	none	CODE

8. Выводы по ЛР № 4

В данной лабораторной работе были выполнены следующие задачи:

- получены навыки разработки и отладки программ на ЯП,
- приобретены знания об использовании циклов и знания о написании методов по переводу символов на Ассемблере.